NAME: PETTYSHAIN KINYA KARIUKI
REG NO. : SCT221-0186/2024

DATA STRUCTURES & ALGORITHMS REPORT

Title: Implementation and Performance Analysis of Searching Algorithms

1. Introduction

This assignment focuses on implementing and comparing three fundamental searching techniques used in Data Structures and Algorithms. The algorithms implemented are Linear Search, Binary Search, and Hashing. The main objective is to develop a menu-driven program that allows a user to select a searching method, execute it, measure execution time, and compare performance results.

2. Objectives

To design a menu-driven searching program
To implement Linear Search, Binary Search, and Hashing
To measure execution time of each algorithm
To compare the efficiency of all three methods

3. Algorithms Implemented

3.1 Linear Search
Linear Search scans each element of the list one by one until the target is found or the list ends.
Works on unsorted data
Simple but inefficient for large datasets
Time Complexity: O(n)
3.2 Binary Search
Binary Search works by dividing a sorted array into halves and repeatedly narrowing the search range.
Requires sorted data
Much faster than linear search
Time Complexity: O(log n)
3.3 Hashing
Hashing stores data in a hash table and allows direct access using keys.
Very fast retrieval
Uses key-value mapping
Time Complexity: O(1) average case

4. Program Code

Python
```
import time

# Linear Search
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Binary Search
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
```

```
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Hashing
def hashing_search(hash_table, key):
    return hash_table.get(key, -1)

# Menu-driven program
def main():
    arr = [10, 20, 30, 40, 50, 60, 70, 80]

    key = int(input("Enter number to search: "))

    while True:
        print("\n--- MENU ---")
        print("1. Linear Search")
        print("2. Binary Search")
        print("3. Hashing")
        print("4. Exit")

        choice = int(input("Enter choice: "))

        if choice == 1:
            start = time.time()
            result = linear_search(arr, key)
            end = time.time()

            print("Result:", result)
            print("Time taken:", end - start)

        elif choice == 2:
            arr.sort()
            start = time.time()
            result = binary_search(arr, key)
            end = time.time()

            print("Result:", result)
            print("Time taken:", end - start)

        elif choice == 3:
            hash_table = {arr[i]: i for i in range(len(arr))}

            start = time.time()
            result = hashing_search(hash_table, key)
            end = time.time()

            print("Result:", result)
            print("Time taken:", end - start)

        elif choice == 4:
            break

        else:
            print("Invalid choice")
```

main()
5. Time Measurement Method
The execution time of each algorithm was calculated using:
Time Taken = End Time − Start Time
This allows comparison of performance under the same input conditions.
6. Sample Input
The program was tested using the following input:
Search value: 40

7. Sample Output

```
PS C:\Users\laure>  & 'C:\Users\laure\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\laure\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-
x64\bundled\libs\debugpy\launcher' '61025' '--' 'c:\Users\laure\importtime.py'
Enter number to search: 40

--- MENU ---
1. Linear Search
2. Binary Search
3. Hashing
4. Exit
Enter choice: 1
Result: 3
Time taken: 5.1975250244140625e-05

--- MENU ---
1. Linear Search
2. Binary Search
3. Hashing
4. Exit
Enter choice: 2
Result: 3
Time taken: 3.218650817871094e-05

--- MENU ---
1. Linear Search
2. Binary Search
3. Hashing
4. Exit
Enter choice: 3
Result: 3
Time taken: 1.621246337890625e-05

--- MENU ---
1. Linear Search
2. Binary Search
3. Hashing
4. Exit
Enter choice: 4
PS C:\Users\laure>
```

8. Performance Comparison

Algorithm
Performance
Time Complexity
Linear Search
Slow

O(n)
Binary Search
Fast
O(log n)
Hashing
Fastest
O(1) average

9. Discussion

From the results, Hashing proved to be the fastest method because it allows direct access using keys. Binary Search performed better than Linear Search because it reduces the search space by half in each step. Linear Search was the slowest since it checks each element sequentially.

10. Conclusion

The program successfully implemented Linear Search, Binary Search, and Hashing using a menu-driven approach. The comparison showed that Hashing is the most efficient method, followed by Binary Search, and finally Linear Search. The experiment demonstrates that the choice of algorithm significantly affects performance.

11. Recommendation

Use Hashing for fast data retrieval
Use Binary Search for sorted datasets
Use Linear Search only for small or unsorted datasets

THE END.